

Práctico AYED1 - Sala 2 - 23-09

Práctico 3 - Ejercicio 8a

Derivar:

$$\begin{aligned} \text{sumaMin} &: [\text{Int}] \rightarrow \text{Int} \\ \text{sumaMin.xs} &= \langle \text{Min as, bs, cs} : \text{xs} = \text{as} ++ \text{bs} ++ \text{cs} : \text{sum.bs} \rangle \end{aligned}$$

Esta función calcula la suma del segmento de suma mínima de xs.

Es decir, considera todos los segmentos posibles de xs, calcula su suma y toma el mínimo.

Derivación: La haremos por inducción en xs.

Caso base: Ejercicio

sumaMin.[]
= { pasos varios }
0

Paso inductivo: $\text{xs} = \text{y} \blacktriangleright \text{ys}$

sumaMin.(y ▶ ys)
= { esp. }
 $\langle \text{Min as, bs, cs} : \text{y} \blacktriangleright \text{ys} = \text{as} ++ \text{bs} ++ \text{cs} : \text{sum.bs} \rangle$
= { nos interesa ver si $\text{as} = [] \vee \text{as} \neq []$ }
 $\langle \text{Min as, bs, cs} : \text{y} \blacktriangleright \text{ys} = \text{as} ++ \text{bs} ++ \text{cs} \wedge (\text{as} = [] \vee \text{as} \neq []) : \text{sum.bs} \rangle$
= { distributividad }

$\langle \text{Min } as, bs, cs : (y \triangleright ys = as ++ bs ++ cs \wedge as = []) \vee (y \triangleright ys = as ++ bs ++ cs \wedge as \neq []) : \text{sum.bs} \rangle$
 = { partición de rango }

$\langle \text{Min } as, bs, cs : y \triangleright ys = as ++ bs ++ cs \wedge as = [] : \text{sum.bs} \rangle_{\text{min}}$

$\langle \text{Min } as, bs, cs : y \triangleright ys = as ++ bs ++ cs \wedge as \neq [] : \text{sum.bs} \rangle$

= { cambio de variable $as \rightarrow a \triangleright as$, estoy aplicando una función biyectiva ya que $as \neq []$ }

$\langle \text{Min } as, bs, cs : y \triangleright ys = as ++ bs ++ cs \wedge as = [] : \text{sum.bs} \rangle_{\text{min}}$

$\langle \text{Min } a, as, bs, cs : y \triangleright ys = \underline{(a \triangleright as) ++ bs ++ cs} \wedge \underline{a \triangleright as \neq []} : \text{sum.bs} \rangle$

= { varias prop. de listas }

$\langle \text{Min } as, bs, cs : y \triangleright ys = as ++ bs ++ cs \wedge as = [] : \text{sum.bs} \rangle_{\text{min}}$

$\langle \text{Min } a, as, bs, cs : \underline{y \triangleright ys = a \triangleright (as ++ bs ++ cs)} \wedge \underline{\text{True}} : \text{sum.bs} \rangle$

= { más prop. de listas }

$\langle \text{Min } as, bs, cs : y \triangleright ys = as ++ bs ++ cs \wedge as = [] : \text{sum.bs} \rangle_{\text{min}}$

$\langle \text{Min } a, as, bs, cs : y = a \wedge ys = as ++ bs ++ cs : \text{sum.bs} \rangle$

= { eliminación de variable a }

$\langle \text{Min } as, bs, cs : y \triangleright ys = as ++ bs ++ cs \wedge as = [] : \text{sum.bs} \rangle_{\text{min}}$

$\langle \text{Min } as, bs, cs : ys = as ++ bs ++ cs : \text{sum.bs} \rangle$

= { Hipótesis Inductiva }

$\langle \text{Min } as, bs, cs : y \triangleright ys = as ++ bs ++ cs \wedge as = [] : \text{sum.bs} \rangle_{\text{min}} \text{ sumaMin.hs}$

= { eliminación de variable as }

$\langle \text{Min } bs, cs : y \triangleright ys = \underline{[] ++ bs ++ cs} : \text{sum.bs} \rangle_{\text{min}} \text{ sumaMin.hs}$

= { prop. listas }

$\langle \text{Min } bs, cs : y \triangleright ys = bs ++ cs : \text{sum.bs} \rangle_{\text{min}} \text{ sumaMin.hs}$

= { Observación: esta expresión cuantificada no la vamos a poder eliminar ni con rango unitario, ni con rango vacío, ni con H.I., ni con nada, es algo que hay que calcular aparte.

Además, vemos que lo que está calculando es como sumaMin pero para segmentos iniciales en lugar de segmentos arbitrarios. Debemos **modularizar**:

$\text{sumaMinInit.hs} = \langle \text{Min } bs, cs : xs = bs ++ cs : \text{sum.bs} \rangle$

```
}
  sumaMinInit.(y ► ys) min sumaMin.ys
```

Listo sumaMin. Resultado parcial:

$\text{sumaMin.[]} \doteq 0$

$\text{sumaMin.}(x \blacktriangleright xs) \doteq \text{sumaMinInit.}(x \blacktriangleright xs) \text{ min sumaMin.xs}$

Falta derivar sumaMinInit: $\text{sumaMinInit.xs} = \langle \text{Min bs, cs : xs} = \text{bs} ++ \text{cs} : \text{sum.bs} \rangle$

Lo hacemos por inducción en xs.

Caso base: Ejercicio.

```
  sumaMinInit.[ ]
= { ????? }
0
```

Paso inductivo: $xs = y \blacktriangleright ys$

```
  sumaMinInit.(y ► ys)
= { esp. }
  ⟨ Min bs, cs : y ► ys = bs ++ cs : sum.bs ⟩
= { me interesa ver si bs = [ ] o no }
  ⟨ Min bs, cs : y ► ys = bs ++ cs ∧ (bs = [ ] ∨ bs ≠ [ ]): sum.bs ⟩
= { distributividad y part. de rango }
  ⟨ Min bs, cs : y ► ys = bs ++ cs ∧ bs = [ ]: sum.bs ⟩ min
  ⟨ Min bs, cs : y ► ys = bs ++ cs ∧ bs ≠ [ ]: sum.bs ⟩
  ⟨ Min bs, cs : y ► ys = bs ++ cs ∧ bs ≠ [ ]: sum.bs ⟩
= { cambio de variable bs → b ► bs, se puede porque bs ≠ [ ] }
  ⟨ Min bs, cs : y ► ys = bs ++ cs ∧ bs = [ ]: sum.bs ⟩ min
```

$\langle \text{Min } b, bs, cs : y \blacktriangleright ys = (b \blacktriangleright bs) ++ cs \wedge +b \blacktriangleright bs \neq [] : \text{sum.}(b \blacktriangleright bs) \rangle$

= { prop. listas }

$\langle \text{Min } bs, cs : y \blacktriangleright ys = bs ++ cs \wedge bs = [] : \text{sum.bs} \rangle_{\text{min}}$

$\langle \text{Min } b, bs, cs : y \blacktriangleright ys = b \blacktriangleright (bs ++ cs) \wedge \text{True} : \text{sum.}(b \blacktriangleright bs) \rangle$

= { prop. listas }

$\langle \text{Min } bs, cs : y \blacktriangleright ys = bs ++ cs \wedge bs = [] : \text{sum.bs} \rangle_{\text{min}}$

$\langle \text{Min } b, bs, cs : y = b \wedge ys = bs ++ cs : \text{sum.}(b \blacktriangleright bs) \rangle$

= { eliminacion de variable b }

$\langle \text{Min } bs, cs : y \blacktriangleright ys = bs ++ cs \wedge bs = [] : \text{sum.bs} \rangle_{\text{min}}$

$\langle \text{Min } bs, cs : ys = bs ++ cs : \text{sum.}(y \blacktriangleright bs) \rangle$

= { def. sum }

$\langle \text{Min } bs, cs : y \blacktriangleright ys = bs ++ cs \wedge bs = [] : \text{sum.bs} \rangle_{\text{min}}$

$\langle \text{Min } bs, cs : ys = bs ++ cs : y + \text{sum.bs} \rangle$

= { nos damos cuenta que podemos distribuir + con min. }

Nota: si no nos damos cuenta, hay que generalizar y sale pero es más complicado. }

$\langle \text{Min } bs, cs : y \blacktriangleright ys = bs ++ cs \wedge bs = [] : \text{sum.bs} \rangle_{\text{min}}$

$(\langle \text{Min } bs, cs : ys = bs ++ cs : \text{sum.bs} \rangle + y)$

= { H.I. }

$\langle \text{Min } bs, cs : y \blacktriangleright ys = bs ++ cs \wedge bs = [] : \text{sum.bs} \rangle_{\text{min}} (\text{sumaMinInit.ys} + y)$

= { eliminación de variable bs }

$\langle \text{Min } cs : y \blacktriangleright ys = [] ++ cs : \text{sum.}[] \rangle_{\text{min}} (\text{sumaMinInit.ys} + y)$

= { prop. listas }

$\langle \text{Min } cs : y \blacktriangleright ys = cs : \text{sum.}[] \rangle_{\text{min}} (\text{sumaMinInit.ys} + y)$

= { rango unitario (siempre preferimos rango unitario a termino constante) }

$\text{sum.}[] \text{ min } (\text{sumaMinInit.ys} + y)$

= { def. sum }

$0 \text{ min } (\text{sumaMinInit.ys} + y)$

Listo! Esto ya es todo “programable” (está en el lenguaje de programación).

Resultado final:

$\text{sumaMin}.[] \doteq 0$

$\text{sumaMin}.(x \blacktriangleright xs) \doteq \text{sumaMinInit}.(x \blacktriangleright xs) \text{ min } \text{sumaMin}.xs$

$\text{sumaMinInit}.[] \doteq 0$

$\text{sumaMinInit}.(x \blacktriangleright xs) \doteq 0 \text{ min } (\text{sumaMinInit}.xs + x)$

Observaciones:

1. Los ejercicios que calculan cosas sobre segmentos arbitrarios (como `sumaMin`) se resuelven habitualmente haciendo una modularización que resuelve el mismo problema pero para segmentos iniciales.
2. Volvemos al paso inductivo de `sumaMin`:
 $\langle \text{Min } bs, cs : y \blacktriangleright ys = bs ++ cs : \text{sum}.bs \rangle \text{min } \text{sumaMin}.ys$

Error típico acá es querer seguir derivando usando 3ro excluído para meter “ $bs = [] \vee bs \neq []$ ”, y después partir rango, etc. Esto **no sirve** ya que no tengo a donde ir, no tengo H.I. que aplicar, y sí o sí tarde o temprano voy a tener que modularizar (mejor temprano que tarde).